

Guía de Aprendizaje: Tuberías Unidireccionales (Pipes)

Malak Sanchez

Monitorías de Sistemas Operativos

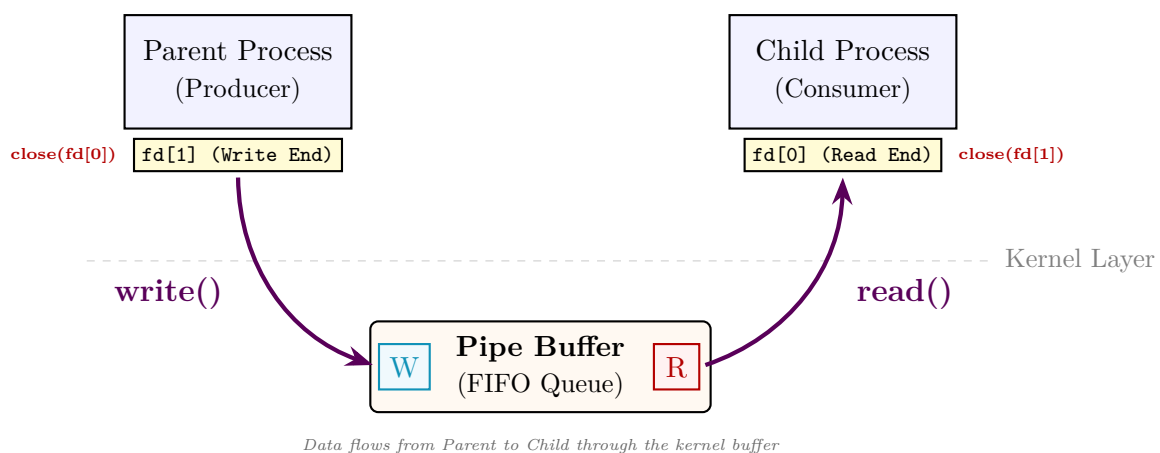
16 de abril de 2026

1. Motivación

Gran parte del poder de la terminal de Linux reside en el operador `|`. Este permite conectar la salida de un proceso con la entrada de otro. Internamente, esto se logra mediante el mecanismo de **Pipes**.

2. Idea Fundamental

Un pipe es un buffer circular en el espacio del kernel que permite el flujo de datos en una sola dirección. Para que dos procesos se comuniquen mediante un pipe, deben tener un ancestro común que haya creado el pipe antes de realizar un `fork()`.



3. Sintaxis Básica

```
#include <unistd.h>
int fd[2];
pipe(fd); // Creates the pipe.
           // fd[0]: Read end
           // fd[1]: Write end
```

Descripción de las operaciones

- **pipe(fd)**: Crea un canal de comunicación y llena el arreglo **fd** con dos descriptores de archivo. Devuelve 0 en éxito y -1 en error.
- **write(fd[1], buf, n)**: Envía **n** bytes desde el buffer **buf** hacia el pipe. Si el pipe está lleno, el proceso se bloquea.
- **read(fd[0], buf, n)**: Lee hasta **n** bytes del pipe y los guarda en **buf**. Si el pipe está vacío, el proceso se bloquea hasta que haya datos.
- **close(fd)**: Cierra un extremo del pipe. Es **crítico** cerrar los extremos no utilizados en cada proceso para que **read()** pueda detectar el final del archivo (EOF) cuando todos los escritores cierran el pipe.

4. Ejemplo: Comunicación Padre-Hijo

En este ejemplo, el padre envía un mensaje al hijo. Note cómo cada proceso cierra el extremo que no va a utilizar inmediatamente después del **fork()**.

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <string.h>
4  #include <sys/wait.h>
5
6  int main() {
7      int fd[2];
8      char buffer[30];
9
10     if(pipe(fd) == -1){
11         perror("pipe");
12         return 1;
13     }
14
15     if(fork() > 0){ // Parent process
16         close(fd[0]); // Parent will not read, close read end
17
18         char msg[] = "Message from your parent!";
19         write(fd[1], msg, strlen(msg) + 1);
20
21         close(fd[1]); // Finished writing
22         wait(NULL);   // Wait for child to finish
23     }else{ // Child process
24         close(fd[1]); // Child will not write, close write end
25
26         read(fd[0], buffer, sizeof(buffer));
27         printf("Child received: %s\n", buffer);
28
29         close(fd[0]); // Finished reading
30     }
31     return 0;
32 }
```

5. Conceptos Avanzados

- **Tuberías con nombre (FIFOs):** A diferencia de los pipes anónimos, los FIFOs tienen un nombre en el sistema de archivos y permiten la comunicación entre procesos sin parentesco.

6. Ejercicios

1. Modifique el ejemplo para que la comunicación sea bidireccional (**Full-Duplex**). ¿Cuántos pipes necesita?
2. Investigue qué sucede si un proceso escribe en un pipe cuyo extremo de lectura ha sido cerrado por todos los procesos (Señal **SIGPIPE**).